ANEXO I – Práctica 4

Implementación de algoritmos genéticos en Matlab®: *Global Optimization Toolbox*

1. *Global Optimization toolbox*

El *Global Optimization toolbox* es el *toolbox*/librería de Matlab para resolver problemas de optimización. En este *toolbox* se integran diferentes enfoques: (i) desde los enfoques tradicionales basados en derivadas y gradientes (*Global or Multiple Starting Point Search*); (ii) pasando por la computación evolutiva mediante los algoritmos genéticos (*Genetic Algorithm*); (iii) hasta otras técnicas también dentro de los enfoques de optimización estocásticos basadas en la observación y "copia" de diferentes procesos naturales, como *Particle Swarm Optimization* y *Simulated Annealing*.

➔ En este Anexo se detallan las funcionalidades de Matlab en torno a los algoritmos genéticos, que serán las herramientas empleadas en la Práctica 4.

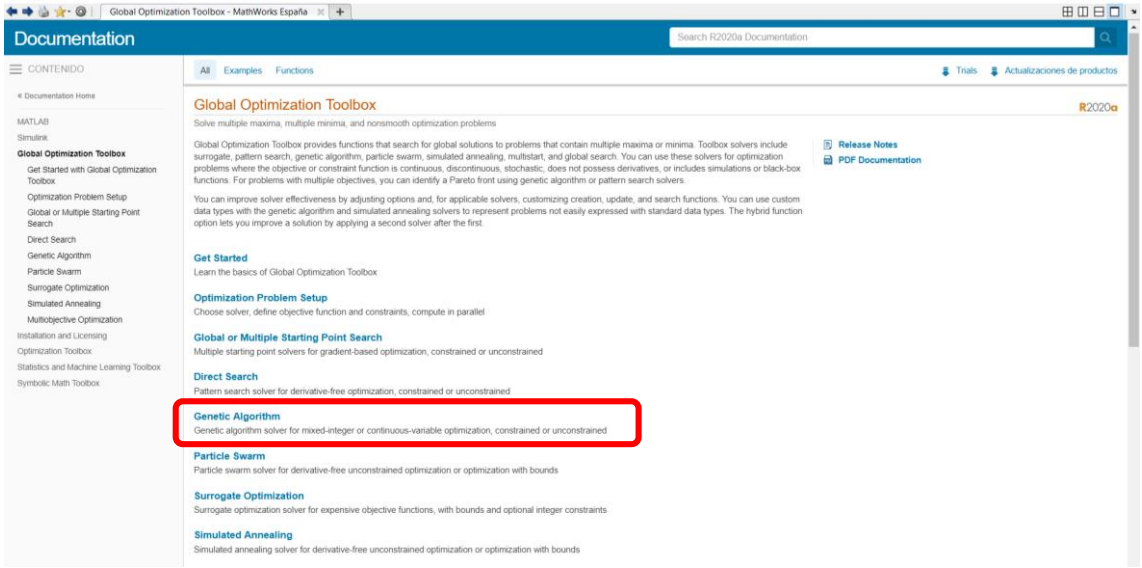

Figura 1. Enfoques de optimización implementados en el *Global Optimization toolbox* de Matlab.

Para trabajar con algoritmos genéticos, este *toolbox* nos ofrece **tres funciones**:

- ***ga***, que es la encargada de resolver el problema de optimización
  - Este es el denominado <u>*solver*</u> (*i.e.*, la función que resuelve el problema de optimización) para este enfoque. Cada enfoque tiene su *solver* particular: 'simulannealbnd' para *Simulated Annealing*; '*particleswarm*' para *Particle Swarm Optimization*; '*GlobalSearch*' para la búsqueda tradicional basada en gradientes, etc.
  - IMP. **Por defecto, *ga* siempre busca <u>minimizar</u> la función objetivo** y esto es algo que no podemos cambiar. Si lo que queremos es maximizar, la tenemos que "engañar", por ejemplo, simplemente cambiando el signo de la función.

- ***optimoptions***, que crea una estructura de datos para pasar diferentes parámetros y sus valores al *solver ga*.

- ***resetoptions***, que permite resetear la estructura de opciones a los valores especificados por defecto.
    - Las funciones *optimoptions* y *resetoptions* son genéricas para todos los enfoques de optimización, por eso, cada vez que usemos *optimoptions* para inicializar un algoritmo genético tendremos que decirle qué esquema de optimización estamos usando indicando el *solver* correspondiente.

2. Sintaxis de la función principal *ga*

Esta es la sintaxis de la función, con todos los parámetros de entrada y de salida que nos permite especificar:

Syntax

```
x = ga(fun,nvars)
x = ga(fun,nvars,A,b)
x = ga(fun,nvars,A,b,Aeq,beq)
x = ga(fun,nvars,A,b,Aeq,beq,lb,ub)
x = ga(fun,nvars,A,b,Aeq,beq,lb,ub,nonlcon)
x = ga(fun,nvars,A,b,Aeq,beq,lb,ub,nonlcon,options)
x = ga(fun,nvars,A,b,[],[],lb,ub,nonlcon,IntCon)
x = ga(fun,nvars,A,b,[],[],lb,ub,nonlcon,IntCon,options)
x = ga(problem)
[x,fval] = ga(___)
[x,fval,exitflag,output] = ga(___)
[x,fval,exitflag,output,population,scores] = ga(___)
```

Figura 2. Sintaxis de la función *ga*.

En su llamada/ejecución más básica/sencilla

x = ga(fun,nvars)

ga busca el mínimo (que será el valor ***x*** que devuelve) de una función objetivo que especificamos mediante el argumento de entrada ***fun***, sin ningún tipo de restricción o límite, es decir, en todo el espacio de búsqueda. ***nvars*** será la dimensionalidad de ese espacio de búsqueda, es decir, el número de variables de la función objetivo.

Si queremos especificar algún tipo de límite sobre *x* para reducir el espacio de búsqueda, podemos introducir restricciones mediante varios de los parámetros de entrada a ***ga***:

- ***A*** y ***b*** permiten especificar inecuaciones/desigualdades lineales del tipo $A \cdot x \leq b$, donde *A* es una matriz de tamaño *M x nvars* y *b* es un vector de longitud *M*, siendo *M* el número de desigualdades que queremos expresar. De esta forma, la región del espacio bidimensional determinada por las siguientes inecuaciones se presentaría así:

$x_1 + 2x_2 \leq 10$ $3x_1 + 4x_2 \leq 20$ $5x_1 + 6x_2 \leq 30$	`A = [1,2;3,4;5,6];` `b = [10;20;30];`

- ***Aeq*** y ***beq*** permite especificar igualdades del tipo *Aeq*x = beq*, donde *Aeq* es una matriz de tamaño *M x nvars* y *beq* es un vector de longitud *M*, siendo *Meq* el número de igualdades que queremos expresar. Por ejemplo, las siguientes ecuaciones lineales se presentarían así:

$x_1 + 2x_2 + 3x_3 = 10$ $2x_1 + 4x_2 + x_3 = 20$	`Aeq = [1,2,3;2,4,1];` `beq = [10;20];`

- ***lb*** y ***ub*** permiten definir un conjunto de límites inferiores (*lower bounds*) y superiores (*upper bounds*), de forma que se limita el espacio de búsqueda a la región *lb* ≤ *x* ≤ *ub*. Tanto *lb* como *ub* son vectores de tamaño *Nvars*. Por ejemplo:

`lb = [0;-Inf;4]` means `x(1) ≥ 0, x(3) ≥ 4`	`ub = [Inf;4;10]` means `x(2) ≤ 4, x(3) ≤ 10`

- ***nonlicon*** permite especificar fronteras no lineales en el espacio de búsqueda (*nonlinear constrains*), pero <u>no vamos a entrar en esto</u> porque el *solver ga* no fuerza a que se cumplan las restricciones no lineales cuando se trabaja con cadenas de bits, que es la forma tradicional de los algoritmos genéticos y la que emplearemos en la Práctica 2.

- ***Intcon*** permite especificar cuáles de las componentes/dimensiones del espacio de búsqueda toma exclusivamente valores enteros. Cada valor en *Intcon* representa una componente del espacio de búsqueda, por lo que será un vector de números enteros (ya que representan componentes/posiciones) y de tamaño máximo *nvars*. Por ejemplo, si por las condiciones de nuestro problema, queremos expresar que las 3 últimas dimensiones son enteras, pondríamos *Intcon* = [*nvars*-2:1:*nvars*];

    - Lo que estamos haciendo (porque la naturaleza de nuestro problema así lo exige) es restringir los valores de las variables/genes de las dimensiones especificadas exclusivamente al conjunto de número enteros. Esto afecta también al tipo de operaciones (mutación y cruce), puesto que no se pueden aplicar "así como así" a cualquier tipo de dato.

- ***options*** <u>será para nosotros el argumento de entrada al algoritmo genético más importante</u>. Es un dato de tipo estructura, que se crea con la función ***optimoptions*** y que permite especificar todos los parámetros libres sobre los que tiene capacidad de decisión el diseñador: la población inicial (*i.e.*, el punto de partida del GA); la estrategia de recombinación y la probabilidad de cruce (P_c); la estrategia de mutación y la probabilidad de mutación (P_m); y diferentes estrategias de parada, como el número máximo de generaciones o tiempos de computación, así como <u>otras muchas opciones de configuración que la mayoría de las veces no se utilizarán y tomarán su valor por defecto</u>. Vamos a ver las más importantes:
    - ***PopulationType.*** Especifica el tipo de datos de entrada a la función de ajuste, es decir, el tipo de datos de los individuos de la población. Las opciones son:

      `'bitstring' | 'custom' | {'doubleVector'}`

      ga ignores all constraints when `PopulationType` is set to `'bitString'` or `'custom'`. See Population Options.

- Cuando se especifica que el tipo de datos es una cadena binaria '*bitstring*', cualquier limitación especificada al espacio de búsqueda es ignorada, ya que no tiene sentido.
- En Matlab la función ***ga*** tiene un **uso mucho más general** que el particular de los algoritmos genéticos, ya que los GAs por definición trabajan con cadenas de bits, mientras que Matlab permite también trabajar con enteros de tipo *double*.
- De hecho, *double* es el valor por defecto. También se pueden especificar tipos de datos customizados.

- ***PopulationSize.*** Especifica el tamaño de la población, es decir, el número de individuos o soluciones potenciales que se hacen iterar en paralelo en cada generación. Las opciones son:

  Positive integer | {50} when `numberOfVariables <= 5`, {200} otherwise | `{min(max(10*nvars,40),100)}` for mixed-integer problems

- ***CreationFcn.*** Es la función que especifica la población inicial, *i.e.*, el punto de partida del algoritmo genético. Hay varias opciones ya implementadas en Matlab entre las que podremos elegir o también especificar una función de inicialización customizada (a la carta), *i.e.*, adaptada a nuestro problema particular. Las opciones son:

  `{'gacreationuniform'}` | `{'gacreationlinearfeasible'}*` | Custom creation function

  - Tanto '*gacreationuniform*' como '*gacreationlinearfeasible*' están <u>pensadas para problemas de optimización numérica</u> en los que se establecen límites al espacio de búsqueda mediante las inecuaciones e igualdades que hemos visto en los puntos anteriores. Ambas producen una población inicial con parte de los individuos concentrados en los límites y el resto distribuidos aleatoriamente con probabilidad uniforme por el espacio de búsqueda.
  - Creación de funciones customizadas:
    - Se hace a través de una llamada a lo que se denomina *function handle*, que se expresa de la forma *@myfun*, donde *myfun* es el nombre de nuestra función particular.
    - Lógicamente, tiene que devolver una población con indiviuos del mismo tipo de datos que el especificado en *PopulationType*.

- ***InitialPopulationMatrix.*** Permite especificar directamente la población inicial. Es la forma más sencilla de hacerlo, de forma adaptada a nuestro problema particular y sin necesidad de crear una función customizada.
  - Será una matriz, con tantas columnas como número de variables (dimensión del espacio de búsqueda) y con un número de filas que como máximo podrá ser *PopulationSize*:
    - Si es igual a *PopulationSize*, estaremos especificando completamente la población inicial.
    - Si es menor, *ga* llamará a *CreationFcn* para completar el resto de individuos de la población de inicio.

- Si no lo especificamos o le pasamos una matriz vacía '[ ]', *ga* llamará por defecto a *CreateFcn*, con lo que al menos una de las dos cosas hay que hacer (pasar la matriz inicial o definir la función).

- ***CrossoverFcn***. Permite especificar la estrategia de recombinación para crear la descendencia/hijos. Hay varias opciones ya implementadas en Matlab entre las que podremos elegir o también especificar una función de inicialización customizada (*i.e.*, adaptada a nuestro problema particular). Las opciones son:

  {'crossoverscattered'} for ga,
  {'crossoverintermediate'}* for gamultiobj |
  'crossoverheuristic' | 'crossoversinglepoint' |
  'crossovertwopoint' | 'crossoverarithmetic' | Custom crossover function

  - Cuando <u>no se definen restricciones</u> al espacio de búsqueda:

'crossoverscattered'	p1 = [a b c d e f g h] p2 = [1 2 3 4 5 6 7 8] and the binary vector is [1 1 0 0 1 0 0 0], the function returns the following child: child1 = [a b 3 4 e 6 7 8]
'crossoversinglepoint'	For example, if p1 and p2 are the parents p1 = [a b c d e f g h] p2 = [1 2 3 4 5 6 7 8] and the crossover point is 3, the function returns the following child. child = [a b c 4 5 6 7 8]
'crossovertwopoint'	p1 = [a b c d e f g h] p2 = [1 2 3 4 5 6 7 8] and the crossover points are 3 and 6, the function returns the following child. child = [a b c 4 5 6 g h]

  - Cuando <u>se definen restricciones</u> al espacio de búsqueda:

'crossoverintermediate'	Los hijos son promedios ponderados de los padres, lo que permite respetar los límites si los padres los respetaban previamente.
'crossoverheuristic'	Crea hijos en la línea que une a ambos padres, una pequeña distancia con respecto al padre con mayor ajuste en la dirección hacia el otro padre.
'crossoverarithmetic'	El hijo es la media aritmética de los padres

  - También se pueden diseñar estrategias de recombinación customizadas. Del mismo modo que con las funciones de creación de la población inicial, se hace a través de la llamada a una *function handle*, que se expresa de la forma *@myfun*, donde myfun es el nombre de nuestra función custimizada de cruce.

  - **<u>IMP</u>**: En la implementación de GAs de Matlab, <u>la recombinación de dos padres solo produce un único hijo</u>, no dos como vimos en la teoría.

- ***CrossoverFraction.*** **<u>NO ES</u>** la probabilidad de cruce (Pc), pero es la forma en que la implementación de los GAs en Matlab decide cuántos individuos (sin contar la élite) van a ser sustituidos por descendientes creados mediante recombinación en la siguiente generación. El valor por defecto es 0.8.
    - Es decir, ***ga*** no deja al azar si una pareja de padres se recombina o no, que era para lo que se empleaba la probabilidad P_c → Los pares seleccionados siempre se van a reproducir y lo que se fija de antemano es la proporción de la población que será sustituida.
- ***EliteCount***. Permite especificar el grado de elitismo, i.e., el número de "mejores" individuos que sobreviven y pasan inalterados a la siguiente generación. Las opciones son:

  Positive integer | `{ceil(0.05*PopulationSize)}` | `{0.05*(default PopulationSize)}` for mixed-integer problems

- ***FitnessScalingFcn***. Permite realizar un escalado de los valores de ajuste "crudos" (*raw*) devueltos por la función de ajuste para adaptar estos valores a los rangos más apropiados con los que trabajan las funciones típicas de selección (de padres y de supervivientes). Las opciones son:

  `{'fitscalingrank'}` | `'fitscalingshiftlinear'` | `'fitscalingprop'` | `'fitscalingtop'` | Custom fitness scaling function

    - ***'fitscalingrank'*** realiza un escalado de forma que el valor de ajuste será proporcional a su ranking (todos los individuos son por tanto previamente ordenados en función de su valor de ajuste crudo), concretamente 1/sqrt(r), donde r es la posición en el ranking.
    - ***'fitscalingprop'*** realiza un escalado proporcional al valor de fitness crudo de cada individuo.
    - ***'fitscalingtop'*** permite especificar un número *n* de "mejores" individuos (o porcentaje de población) a los que se asignará un valor de fitness de 1/*n* igual para todos ellos, mientras que al resto se les asigna directamente un valor nulo.
    - ***'fitscalingshiftlinear'*** realiza un escalado en función del valor promedio de fitness de toda la población, de forma que el ajuste del "mejor" individuo será igual a una constante (por defecto igual a 2) por el valor medio de la población y, a partir de este valor, el fitness del resto se determina de forma lineal.
    - También se pueden diseñar estrategias de escalado customizadas. Del mismo modo que con las funciones de creación de la población inicial, se hace a través de la llamada a una *function handle*, que se expresa de la forma *@myfun*, donde *myfun* es el nombre de nuestra función customizada de escalado.
- ***MutationFcn***. Permite especificar la estrategia de mutación que se aplicará a los hijos. Las opciones son:

  `{'mutationgaussian'}` for `ga`, `{'mutationadaptfeasible'}*` for `gamultiobj` | `'mutationuniform'` | Custom mutation function

- ***'mutationgaussian'*** es la opción por defecto pero que solo tiene sentido en problemas no binarios. Lo que hace es sumar a cada gen/dimensión del individuo un número aleatorio derivado de una distribución gaussiana de media 0 y desviación estándar 1 controlada por los parámetros `Scale` (controla la diversidad de la mutación en la primera generación) y `Shrink` (controla el decaimiento de la diversidad de la mutación en las sucesivas generaciones) sobre el rango de la población especificado en '*InitialPopulationRange*'. Los valores de `Scale` y `Shrink` por defecto son 1. Para cambiarlos habría que hacerlo a través de las opciones del solver: `options = optimoptions('ga','MutationFcn',{@mutationgaussian, scale, shrink})`
- ***'mutationuniform'*** muta cada gen/dimensión de un individuo con una probabilidad `rate` (sería el P_m que hemos visto en la teoría). El valor de cada gen seleccionado para mutar se sustituye por un número aleatorio seleccionado con probabilidad uniforme dentro del rango de valores. El valor por defecto de la probabilidad `rate` es 0.01. Para cambiarlo habría que hacerlo a través de las opciones del solver: `options = optimoptions('ga','MutationFcn', {@mutationuniform, rate})`
- ***'mutationadaptfeasible'*** es la opción por defecto cuando se establecen límites a las dimensiones/variables para reducir el espacio de búsqueda. Produce mutación en direcciones seleccionadas aleatoriamente sin romper los límites y restricciones especificados.
- También se pueden diseñar estrategias de escalado customizadas. Del mismo modo que con las funciones de creación de la población inicial, se hace a través de la llamada a una *function handle*, que se expresa de la forma *@myfun*, donde *myfun* es el nombre de nuestra función customizada de mutación.

- ***PlotFcn***. Permite especificar diferentes opciones de representación de la evolución del algoritmo genético, es decir, de la función de ajuste a través de las generaciones. Las opciones son:

```
ga or gamultiobj: {[]} | 'gaplotdistance' |
'gaplotgenealogy' | 'gaplotselection' |
'gaplotscorediversity' |'gaplotscores' |
'gaplotstopping' | 'gaplotmaxconstr' | Custom plot
function

ga only: 'gaplotbestf' | 'gaplotbestindiv' |
'gaplotexpectation' | 'gaplotrange'
```

Vamos a centrarnos en las opciones de la función `ga`, que es la que emplearemos en la asignatura (`gamultiobj` hace referencia a multiobjetivo, es decir, un problema de optimización de múltiples funciones objetivo/ajuste, donde típicamente la mejora en una de las dimensiones objetivo implica el empeoramiento de la otra, por lo que hay que buscar una "solución de compromiso". **ESTO NO LO VAMOS A UTILIZAR**)

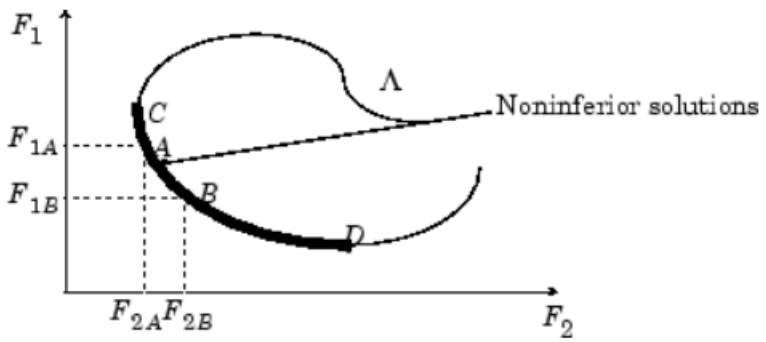

Figura 3. Problema de optimización con 2 funciones objetivo a optimizar (F_1 y F_2). Se muestra el conjunto de puntos (intervalo [C, D]) en que todos los pares son soluciones de no inferioridad (lo que ganas optimizando F_1 lo pierdes de F_2 y viceversa)

- ***'gaplotbestf'*** representa, para cada generación, el valor de ajuste del mejor individuo y el promedio de toda la población.
- ***'gaplotbestindiv'*** representa, para cada generación, al individuo (los valores de sus dimensiones/genes) con mejor ajuste.
- ***'gaplotexpectation'*** representa, para cada generación, el número esperado de hijos *vs.* a los valores de ajuste crudos (*raw*)
- ***'gaplotrange'*** representa, para cada generación, los valores de ajuste mínimo, máximo y promedio.
- También se pueden diseñar funciones de representación de la evolución customizadas. Del mismo modo que con las funciones de creación de la población inicial, se hace a través de la llamada a una *function handle*, que se expresa de la forma *@myfun*, donde *myfun* es el nombre de nuestra función customizada particular.
- También es posible dibujar varias representaciones simultáneamente especificando un cell-array de funciones/opciones de representación: `options = optimoptions('ga','PlotFcn', {@plotfun1, @plotfun2, ...});`

- ***PlotInterval***. Permite especificar el número de generaciones entre cada llamada a las funciones de representación, para no representar en todas las generaciones y que el gráfico no quede muy recargado.
- ***SelectionFcn***. Permite especificar diferentes estrategias de selección de individuos, tanto (i) padres para el proceso de recombinación, como (ii) hijos para el proceso de mutación. Opciones implementadas en el *solver* **ga** de Matlab:

  `{'selectionstochunif'}` for ga, `{'selectiontournament'}` for gamultiobj | `'selectionremainder'` | `'selectionuniform'` | `'selectionroulette'` | Custom selection function

  - ***'selectionstochunif'*** La selección de padres se basa en la representación de una línea imaginaria que es dividida en segmentos/secciones, donde cada segmento corresponde a un individuo y su longitud es proporcional al valor de ajuste (escalado). El algoritmo se mueve a lo

largo de la línea en pasos de igual longitud, de forma que en cada paso se selecciona un individuo.

- El primer paso se determina mediante un número aleatorio menor que la longitud del paso y con probabilidad uniforme.

- ***'selectionremainder'*** Parte de los padres se determina de forma determinista, a través de la parte entera del valor de ajuste y el resto de los padres se determina de forma proporcional a la parte decimal del ajuste mediante el método de la ruleta.
- ***'selectionuniform'*** Selecciona los padres de forma aleatoria en base a una distribución de probabilidad uniforme, donde todos los individuos tienen la misma probabilidad.
- ***'selectionroulette'*** Selecciona los padres mediante el método de la ruleta, donde el área de la ruleta asignada a cada individuo es proporcional al valor del ajuste/*fitness*.
- ***'selectiontournament'*** Selecciona los padres mediante el método de torneo, donde el número de individuos seleccionados en cada ronda/torneo se puede especificar. Por defecto su valor es 4. Si lo queremos modificar debemos hacerlo mediante las opciones: `options = optimoptions('ga','SelectionFcn',{@selectiontournament,size})`
- También se pueden diseñar estrategias de selección customizadas. Del mismo modo que con las funciones de creación de la población inicial, se hace a través de la llamada a una *function handle*, que se expresa de la forma *@myfun*, donde *myfun* es el nombre de nuestra función particular de selección de individuos.

- Criterios de parada. Es posible definir múltiples criterios de parada. El *solver* **ga** implementa los siguientes, que pueden especificarse a través de la estructura de opciones:

Opciones para especificar criterios de parada	**Funcionamiento**
FitnessLimit	El algoritmo para si se alcanza este valor de ajuste
FunctionTolerance	El algoritmo para si el valor promedio del fitness máximo almacenado en las últimas `MaxStallGenerations` generaciones es inferior al valor especificado en `FunctionTolerance`. Por defecto es 10^{-6}
MaxGenerations	El algoritmo para al llegar a este número de iteraciones si no ha parado antes por otro motivo. El valor por defecto es 100 veces el número de variables (dimensión del espacio de búsqueda)

MaxStallGenerations	Permite especificar el número de generaciones en las que se calcula el incremento de fitness para evaluar el criterio de parada de la tolerancia `FunctionTolerance`
MaxStallTime	El algoritmo genético para si no se produce una mejora en la función objetivo durante el tiempo especificado por este parámetro medido en segundos. Por defecto está fijado a infinito {`Inf`}
MaxTime	El algoritmo para si se sobrepasa este tiempo de ejecución a nivel de iteración (no de ejecución global). Se evalúa una vez finalizada la iteración, para permitir completar el cálculo de esa generación. Por defecto es infinito {`Inf`}

- El argumento de entrada ***problem*** no es más que una estructura de datos que define un problema de optimización particular especificando todos los valores de los argumentos de entrada al *solver* **ga**:

`fitnessfcn`	Fitness functions
`nvars`	Number of design variables
`Aineq`	A matrix for linear inequality constraints
`Bineq`	b vector for linear inequality constraints
`Aeq`	Aeq matrix for linear equality constraints
`Beq`	beq vector for linear equality constraints
`lb`	Lower bound on x
`ub`	Upper bound on x
`nonlcon`	Nonlinear constraint function
`rngstate`	Optional field to reset the state of the random number generator
`solver`	`'ga'`
`options`	Options created using `optimoptions` or exported from the Optimization app

Figura 4. Estructura *problem* de entrada al *solver* **ga** que especifica todas las particularidades de un problema bajo estudio.

Esta estructura se crea automáticamente a través de la *Optimization App*, que es una especie de *benchmark* (*i.e.*, banco de pruebas) para dar los primeros pasos/probar los diferentes esquemas de optimización que ofrece el *Global Optimization toolbox*.

➔ **<u>NO</u> LO VAMOS A UTILIZAR EN LA ASIGNATURA**

3. Paso de opciones al *solver* de optimización

Para crear un objeto con la estructura de opciones e inicializar/especificar los parámetros y opciones típicas de los algoritmos genéticos se debe utilizar la función *optimoptions*. Un ejemplo de inicialización sería el siguiente:

```
options=optimoptions('ga',...
    'InitialPopulationMatrix',IP,...
    'CrossoverFraction',Pc(i),...
    'CrossoverFcn',@crossoversinglepoint,...
    'EliteCount',Elite(k),...
    'FitnessScalingFcn',@fitscalingprop,...
    'MaxGenerations',NGen,...
    'PopulationType','bitstring',...
    'MutationFcn',{@mutationuniform,Pm(j)},...
    'PlotFcn',{@gaplotbestf,fPlotBest,fPlotNvars},...
    'PlotInterval',1,...
    'PopulationSize',PopulationSize,...
    'SelectionFcn',@mySelectionFcn_roulette,...
    'MaxStallGenerations',50,...
    'MaxStallTime',Inf,...
    'MaxTime',Inf);
```

Figura 5. Ejemplo de inicialización de un GA mediante *optimoptions*.

En primer lugar, se especifica el *solver* que vamos a emplear en el problema de optimización (en este caso '**ga**'), para que la función sepa cuál es la estructura de parámetros que debe crear. A continuación, se especifican los parámetros configurables mediante el par 'Nombre', valor.

Si visualizamos el objeto creado ejecutándolo en línea de comandos podremos ver todas las opciones configurables, que se diferenciarán entre las que hemos configurado nosotros (*Set Properties*) y las que hemos dejado a sus valores por defecto (*Default Properties*):

```
Command Window
options =

  ga options:

   Set properties:
                    CrossoverFcn: @crossoversinglepoint
              CrossoverFraction: 0.8000
                     EliteCount: 1
              FitnessScalingFcn: @fitscalingprop
        InitialPopulationMatrix: [30×126 double]
                  MaxGenerations: 100
            MaxStallGenerations: 50
                   MaxStallTime: Inf
                        MaxTime: Inf
                    MutationFcn: {@mutationuniform  [0.0100]}
                        PlotFcn: {1×3 cell}
                  PopulationSize: 100
                  PopulationType: 'bitstring'
                    SelectionFcn: @mySelectionFcn_roulette

   Default properties:
            ConstraintTolerance: 1.0000e-03
                    CreationFcn: @gacreationuniform
                        Display: 'final'
                    FitnessLimit: -Inf
              FunctionTolerance: 1.0000e-06
                       HybridFcn: []
         InitialPopulationRange: []
            InitialScoresMatrix: []
  NonlinearConstraintAlgorithm: 'auglag'
                       OutputFcn: []
                    UseParallel: 0
```

Figura 6. Parámetros configurables del *solver* ***ga***.

En cuanto a los parámetros de salida, el *solver* **ga** ofrece las siguientes opciones:

```
[x,fval] = ga(___)
[x,fval,exitflag,output] = ga(___)
[x,fval,exitflag,output,population,scores] = ga(___)
```

Figura 7. Posibles salidas de la función de optimización mediante GAs.

- *x*. Es la solución al problema de optimización, es decir, el individuo con el mayor valor de ajuste en la población al finalizar las iteraciones, típicamente un vector de dimensión *nvars*.
- *fval*. Es el valor de ajuste/*fitness*/*score* del mejor individuo/solución, típicamente un número real.
- *exitflag*. Nos informa sobre la razón por la cual el algoritmo genético ha finalizado. Se muestra como un número entero con la siguiente codificación:

Exit Flag	Meaning
1	**Without nonlinear constraints** — Average cumulative change in value of the fitness function over `MaxStallGenerations` generations is less than `FunctionTolerance`, and the constraint violation is less than `ConstraintTolerance`.
	With nonlinear constraints — Magnitude of the complementarity measure (see Complementarity Measure) is less than `sqrt(ConstraintTolerance)`, the subproblem is solved using a tolerance less than `FunctionTolerance`, and the constraint violation is less than `ConstraintTolerance`.
3	Value of the fitness function did not change in `MaxStallGenerations` generations and the constraint violation is less than `ConstraintTolerance`.
4	Magnitude of step smaller than machine precision and the constraint violation is less than `ConstraintTolerance`.
5	Minimum fitness limit `FitnessLimit` reached and the constraint violation is less than `ConstraintTolerance`.
0	Maximum number of generations `MaxGenerations` exceeded.
-1	Optimization terminated by an output function or plot function.
-2	No feasible point found.
-4	Stall time limit `MaxStallTime` exceeded.
-5	Time limit `MaxTime` exceeded.

Figura 8. Codificación de los criterios de parada.

- *output*. Proporciona una estructura de datos con información sobre el proceso de optimización:

Information about the optimization process, returned as a structure with these fields:

- `problemtype` — Problem type, one of:
  - `'unconstrained'`
  - `'boundconstraints'`
  - `'linearconstraints'`
  - `'nonlinearconstr'`
  - `'integerconstraints'`
- `rngstate` — State of the MATLAB random number generator, just before the algorithm started. You can use the values in `rngstate` to reproduce the output of `ga`.
- `generations` — Number of generations computed.
- `funccount` — Number of evaluations of the fitness function.
- `message` — Reason the algorithm terminated.
- `maxconstraint` — Maximum constraint violation, if any.

Figura 9. Información sobre la evolución del GA.

- *population*. Es la población final, la que se mantiene en la última generación. Será una matriz de tamaño *PopulationSize* × *nvars*, y cada fila representará a un individuo.
- *scores*. Son los valores de ajuste/*fitness* de la población final. Serán por tanto los valores de ajuste del individuo correspondiente a cada fila de la matriz *population*, y será por tanto un vector columna.